CS 240: Artificial Intelligence & Machine Learning Lab

Social User Classification using GCNs on a Social Network

23B1011 23B0902

April 2025

Introduction

Graph Neural Networks (GNNs) have emerged as powerful tools for learning on graph-structured data, enabling applications in social networks, biological systems, and recommendation engines. Among GNN variants, Graph Convolutional Networks (GCNs) offer a scalable and efficient means of propagating information across nodes by aggregating feature information from neighbors. This project investigates the application of GCNs on a real-world social graph dataset to classify users based on their interaction patterns and profile features. Specifically, the project uses a Twitch social graph and explores classification by language and the potential for bot detection. The broader aim is to evaluate the effectiveness of GCNs in social media contexts and compare their performance with other GNN variants such as GraphSAGE and Graph Attention Networks (GAT).

Objectives

- Implement a Graph Convolutional Network (GCN) for node classification using the PyTorch Geometric framework.
- Preprocess the dataset by constructing a graph with appropriate edge lists, node features, and class labels.
- Train and evaluate the GCN model on classification tasks (e.g., language prediction).
- Compare the performance of GCN with alternative models like GraphSAGE and GAT.
- Visualize node embeddings using dimensionality reduction techniques such as t-SNE or UMAP.
- Explore extensions including feature augmentation and preliminary bot detection based on graph structure and activity features.

Dataset

- **Source:** Stanford Network Analysis Project (SNAP)
- **Reference Paper:** Evaluating Proximity Preserving and Structural Role-based Node Embeddings
- **Access:** SNAP Datasets - Social Networks Section
- **Dataset Description:** The Twitch social graph contains user interactions with node-level attributes such as language and activity. Each edge denotes a directed relationship (e.g., followership). The dataset is sparse, directed, and includes metadata useful for classification.

- **Work Directory:** Project Drive Folder

Methodology

1. **Dataset Preprocessing:** Download the raw dataset and parse it into an edge list format. Normalize and encode node features and labels (e.g., language as categorical variables).
2. **Graph Construction:** Use PyTorch Geometric to build a `Data` object containing the edge index (COO format), node features matrix `x`, and label vector `y`. Split data into training, validation, and test sets.
3. **GCN Model Development:** Implement a multi-layer GCN model using `GCNConv` layers, ReLU activations, and dropout for regularization. Employ cross-entropy loss and the Adam optimizer.
4. **Training and Evaluation:** Train the model for a fixed number of epochs and evaluate performance using classification accuracy and F1-score on the test set.
5. **Visualization:** Reduce high-dimensional node embeddings to 2D using t-SNE or UMAP. Visualize clusters to interpret how well the GCN separates user classes.
6. **Model Comparison:** Implement and evaluate GraphSAGE and/or GAT on the same task. Compare results with the GCN model to understand trade-offs in complexity and accuracy.
7. **Extension:** Investigate simple heuristics for bot detection using structural features such as in-degree, out-degree, clustering coefficient, and activity. Apply unsupervised techniques for anomaly detection or binary classification using GCN outputs.

Expected Outcomes

- A well-trained GCN capable of accurately classifying users based on profile and interaction features.
- Quantitative comparison of GCN, GraphSAGE, and GAT performance on the same classification task.
- 2D visualizations of node embeddings showing clustering by language or role.
- A prototype framework for graph-based bot detection that utilizes structural anomalies and GCN outputs.

Timeline

Week	Task
Checkpoint	Dataset setup
Week 1	Basic analysis, implementing model on a smaller dataset
Week 2	GCN model implementation and training, evaluation, visualization, and model comparison
Week 3	Extension (bot detection, feature engineering), report writing